License Hunter - Full Conversation

The complete implementation discussion, preserved from the supplied conversation text and color-coded for sharing.

You're asking the right question. If the user's request is only text, the agent cannot magically know whether a license exists. The **source marketplace must be the authority**.

The product works only when you connect it to providers that expose:

1. Search/catalog data
2. Actual license terms
3. Current price and availability
4. A purchase or checkout operation
5. A downloadable receipt/license certificate

Several stock providers already expose pieces of this. Adobe Stock has APIs for checking licensing options, member entitlements, licensing an asset, and downloading it. Shutterstock's API supports searching and licensing media, and MotionElements advertises an API for searching, purchasing, and downloading stock video/audio. Adobe licensing API (https://developer.adobe.com/stock/docs/api/12-licensing-reference), Shutterstock API (https://api-reference.shutterstock.com/), MotionElements API (https://api-docs.motionelements.com/).

The actual workflow

1. The user's text becomes requirements

The user says:

> "Find music for a worldwide paid YouTube ad, perpetual use, under $50, no attribution."

Your system converts that into structured requirements:

```
{
  "asset_type": "music",
  "commercial_use": true,
  "paid_ads": true,
  "territory": "worldwide",
  "duration": "perpetual",
  "attribution": false,
  "budget": 50
}
```

The text is only the request. It is not evidence.

2. The agent searches real provider catalogs

Your connector calls a provider's API:

```
search_music(
  query="modern fintech background",
  provider="Provider A"
)
```

The provider returns real asset IDs:

```
{
  "asset_id": "track_88421",
  "title": "Future Motion",
  "price": 29,
  "available_license_types": ["commercial", "advertising"],
  "license_url": "https://provider.com/license/track_88421"
}
```

The agent should never invent an asset or license. Every candidate must have a provider ID and a provider URL.

3. The agent asks the provider what license is actually available

For every candidate, call a license-information endpoint:

```
get_license_options(asset_id="track_88421")
```

That should return something like:

```
{
  "asset_id": "track_88421",
  "license_options": [
    {
      "type": "commercial_standard",
      "price": 29,
      "territories": ["worldwide"],
      "duration": "perpetual",
      "paid_advertising": true,
      "attribution_required": false
    },
    {
      "type": "personal",
      "price": 9,
      "territories": ["worldwide"],
      "duration": "perpetual",
      "paid_advertising": false
    }
  ]
}
```

Now the agent can reject the $9 option because it does not permit advertising.

Adobe's API explicitly distinguishes license states and license types, and lets an application check whether an asset is already licensed or whether the user has purchasing quota. That is the kind of provider integration you need. Adobe licensing reference (https://developer.adobe.com/stock/docs/api/12-licensing-reference).

4. The rules engine matches the requirements

Do not let the model make the final decision from prose. Use deterministic rules:

```
def compatible(license, req):
    return (
        license["price"] <= req["budget"]
        and license["paid_advertising"] == req["paid_ads"]
        and req["territory"] in license["territories"]
        and license["duration"] >= req["duration"]
        and license["attribution_required"] == req["attribution"]
    )
```

The model can interpret messy license wording, but the final approval should be based on explicit fields.

5. Availability is checked immediately before purchase

Availability can change. So the system does:

```
search
→ get license options
→ user approves asset and budget
→ re-check price and availability
→ purchase
→ download receipt and asset
```

The asset is not considered secured until the provider returns a successful purchase response.

A valid purchase response might look like:

```
{
  "purchase_id": "purchase_93711",
```

```
  "asset_id": "track_88421",
  "license_type": "commercial_standard",
  "amount_paid": 29,
  "currency": "USD",
  "license_document_url": "...",
  "download_url": "...",
  "purchased_at": "2026-07-15T14:20:00Z"
}
```

6. The agent stores proof

The final product is not just an MP3 or image. It is a rights package:

```
project-rights-pack/
  track_88421.mp3
  purchase_receipt.pdf
  license_certificate.pdf
  license_terms_snapshot.html
  attribution.txt
  rights-summary.json
```

The `rights-summary.json` should say:

```
{
  "asset": "Future Motion",
  "provider": "Provider A",
  "purchased_license": "commercial_standard",
  "allowed_use": ["YouTube advertising", "Instagram advertising"],
  "territory": "worldwide",
  "duration": "perpetual",
  "attribution_required": false,
  "purchase_id": "purchase_93711"
}
```

Three ways to implement purchasing

Option A: Real provider API

Best option.

You integrate with a provider that lets your application search, license, and download assets. Adobe, Shutterstock, and MotionElements are examples of providers with API-based licensing capabilities. Shutterstock licensing API (https://api-reference.shutterstock.com/), MotionElements API (https://api-docs.motionelements.com/).

Option B: Checkout handoff

If a provider supports search but not programmatic checkout, your agent returns the exact asset and purchase link.

The user clicks “Buy,” completes payment, and uploads the receipt. Your agent then verifies and stores the license.

This is less autonomous but completely feasible.

Option C: Your own licensed catalog

For the hackathon, this may be the smartest route.

Partner with 10–20 independent musicians, photographers, designers, and footage creators. Each contributor uploads:

- asset
- price
- allowed uses
- territories
- duration

- attribution requirement
- prohibited uses

Your agent owns the checkout and issues the license instantly.

That gives you a fully working purchase flow without waiting for large marketplaces to approve API access.

The realistic hackathon MVP

Do not support music, photos, fonts, video, datasets, and software all at once.

Build:

> **License Hunter for commercial music in video ads**

Use one of these approaches:

- one real stock-music API
- one direct catalog of independent musicians
- or one x402-compatible licensing provider

There is even an AI-agent-focused stock licensing service advertising an x402 flow where an agent can pay for a clip and receive the license plus download URL programmatically. Stockfilm agent API (https://stockfilm.com/for-ai-agents).

Your demo:

1. User asks for music for a paid global ad.
2. Agent searches a real catalog.
3. It displays three tracks and their exact license terms.
4. It rejects one because paid advertising is prohibited.
5. User approves one.
6. Agent pays.
7. Provider returns the asset and license certificate.
8. Agent produces a rights pack.

That is feasible because every important claim comes from a real provider response—not from the user's text and not from the model's imagination.

The core rule is:

> **The agent may interpret the request, but only the licensor can prove that the license exists.**